

VISIÓN POR COMPUTADORA · ARQUITECTURA DE SOFTWARE

Recreando un sistema de videovigilancia con IA junto a Claude Opus 5.5

Objetivo, arquitectura y los trade-offs que estamos evaluando, con cámaras de tránsito en vivo de California.

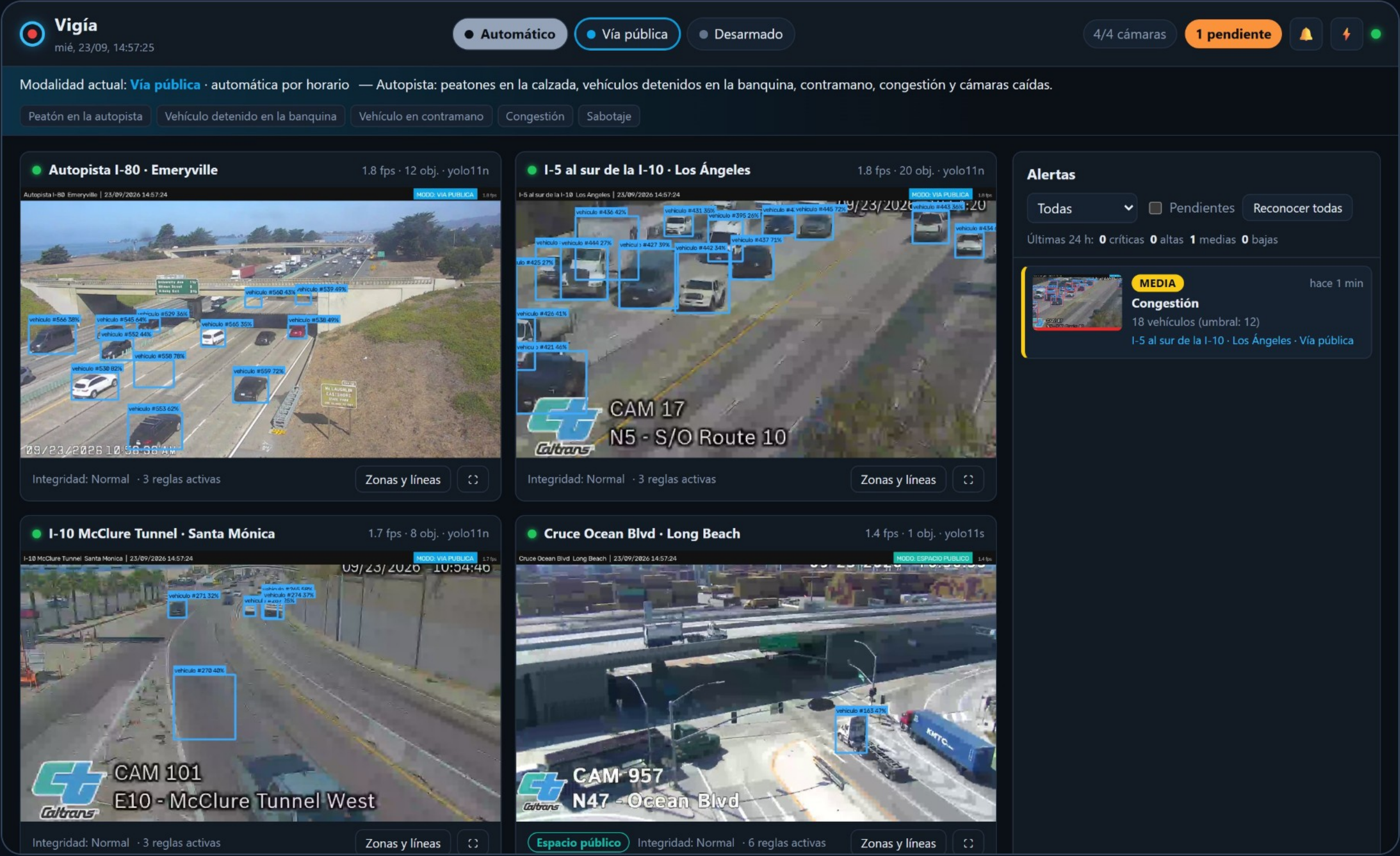


Fig. 1 Vigía analizando en vivo cuatro cámaras públicas de California, 23/09/2026.

OBJETIVO

Detectar comportamientos sospechosos en video, en tiempo real

Qué se considera sospechoso depende de la modalidad activa: En casa, Nocturno, Ausente, Comercio o Vía pública.

RESTRICCIONES DE DISEÑO

Procesamiento local

El video no sale del equipo; no depende de servicios en la nube.

Sin reconocimiento facial

Detecta categorías y comportamientos, no identifica personas.

Hardware modesto

Un portátil con Intel Core i5 de 4 núcleos, sin GPU.

Evidencia en cada alerta

Captura y clip con los segundos previos y posteriores.

Cada decisión de arquitectura se evalúa contra estas cuatro restricciones.

MÉTODO DE TRABAJO

Una colaboración con roles definidos

Yo

Dirección y criterio

- Defino el objetivo y las restricciones
- Elijo qué probar y en qué condiciones
- Tomo la decisión en cada trade-off

Claude Opus 5.5

Diseño, implementación y medición

- Propone la arquitectura y sus alternativas
- Escribe el código y las pruebas
- Mide antes y después de cada cambio

CICLO DE CADA CAMBIO



98

pruebas automáticas

11

reglas de comportamiento

7

modalidades

≈8.800

líneas de código y pruebas

ARQUITECTURA

Un hilo por cámara y siempre el último cuadro

Si la inferencia es más lenta que la cámara, se descartan cuadros en vez de acumular retraso.



OpenCV y PyTorch liberan el GIL durante el cálculo pesado: por eso los hilos sí trabajan en paralelo.

PRUEBA EN CONDICIONES REALES

Probado con cámaras públicas de tránsito, en vivo



Fig. 2 Alerta real en la I-5 (Los Ángeles): congestión de 18 vehículos con umbral de 12, registrada con captura y clip como evidencia.

CÁMARAS	4 transmisiones HLS públicas de Caltrans: Emeryville, Los Ángeles, Santa Mónica y Long Beach
---------	--

EQUIPO	Portátil Intel Core i5-11300H, sin GPU
--------	--

PRUEBA PREVIA	16 alertas reales en 17 minutos: congestión, cámara obstruida y cámara sin señal
---------------	--

TRADE-OFF 1 · PRECISIÓN FRENTE A COSTO

¿Un modelo más grande para todas las cámaras?



Fig. 3 Cruce de Ocean Blvd, Long Beach: camiones vistos desde arriba (YOLO11s).

CRITERIO	YOLO11N	YOLO11S
Detecciones en el cruce	1×	3×
Uso de CPU	1×	2×
Autopista, de noche	Referencia	Sin mejora

DECISIÓN

YOLO11n por defecto; YOLO11s solo en el cruce.

TRADE-OFF 2 · SENSIBILIDAD FRENTE A FALSAS ALARMAS

Detectar más vehículos sin inventar vehículos

De noche, bajar el umbral de confianza de 0,40 a 0,25 pasó de menos de 1 a 3-4 autos detectados por cuadro. Pero un cartel verde y una baliza empezaron a contarse como vehículos.

SOLUCIÓN: UN FILTRO EN TRES CAPAS

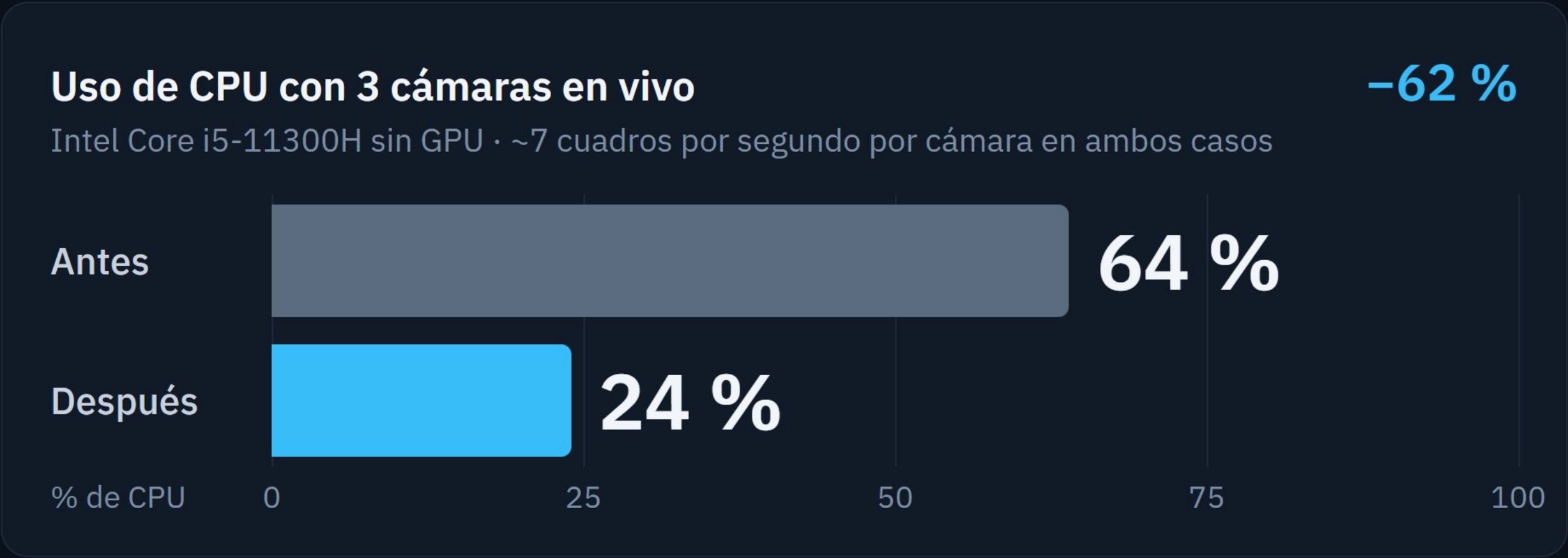
- 1 Lo que nunca se movió no se cuenta.
- 2 Lo quieto y dudoso durante 30 s pasa a ser «decorado».
- 3 Zonas de exclusión dibujadas sobre la imagen.

RESULTADO CON LA MISMA GRABACIÓN DE 3 MINUTOS

MÉTRICA	ANTES	DESPUÉS
Cartel contado como vehículo	94 cuadros	Destellos de 1 cuadro
Autos rápidos cercanos marcados	59 %	79 %

TRADE-OFF 3 · SIMPLICIDAD FRENTE A RENDIMIENTO

Hilos por cámara: simples, hasta que medimos la CPU



DIAGNÓSTICO

Cada cámara ejecuta su propia inferencia y los hilos de PyTorch/OpenMP quedaban en espera activa entre una y otra.

CAMBIOS

- Hilos en reposo entre inferencias: `KMP_BLOCKTIME=1` y `OMP_WAIT_POLICY=PASSIVE`
- Menos hilos por inferencia: núcleos ÷ 4.
- Detección desacoplada del análisis y modo reposo con la escena quieta.

PRÓXIMOS PASOS

Los trade-offs que siguen abiertos

Tracker propio frente a ByteTrack o DeepSORT

EN EVALUACIÓN

Liviano y sin dependencias, pero sin reidentificación por apariencia: una oclusión larga puede cambiar el ID.

Detecciones por segundo frente a vehículos rápidos

EN EVALUACIÓN

A 4 detecciones por segundo, un auto cercano cruza la imagen en 1 o 2 s y a veces se marca tarde.

PyTorch frente a ONNX u OpenVINO

PENDIENTE

Medir cuánto se gana en CPU antes de cambiar el motor de inferencia.

Heurística frente a estimación de pose

PENDIENTE

La regla de altercado es liviana pero experimental; un modelo de pose daría más señal a costa de CPU.

Precisión y recall

PENDIENTE

Evaluar con videos etiquetados (CAVIAR, PETS 2006); hoy no hay métricas formales.

Ya decidido: SSE en lugar de WebSockets (el flujo va en un solo sentido), SQLite local y video al navegador por long-polling de JPEG.

CONCLUSIONES

Lo que aprendimos al salir del entorno de prueba

Las cámaras reales fallan de formas que un video local no muestra.

Transmisiones HLS que llegan en ráfagas, imágenes grises y cortes de señal: varios de esos casos hoy son pruebas automáticas.

Medir antes de optimizar.

El costo de los hilos solo apareció al medir con 3 cámaras en vivo.

Cada decisión, respaldada por un dato.

Opus 5.5 propone y mide; la decisión final se toma con los números a la vista.

ALCANCE

No entrené un modelo (uso YOLO11 preentrenado), no está en producción y todavía no medí precisión ni recall.

¿Qué trade-off habrías resuelto distinto?

Python · YOLO11 · OpenCV · PyTorch · FastAPI · SSE · SQLite · JavaScript